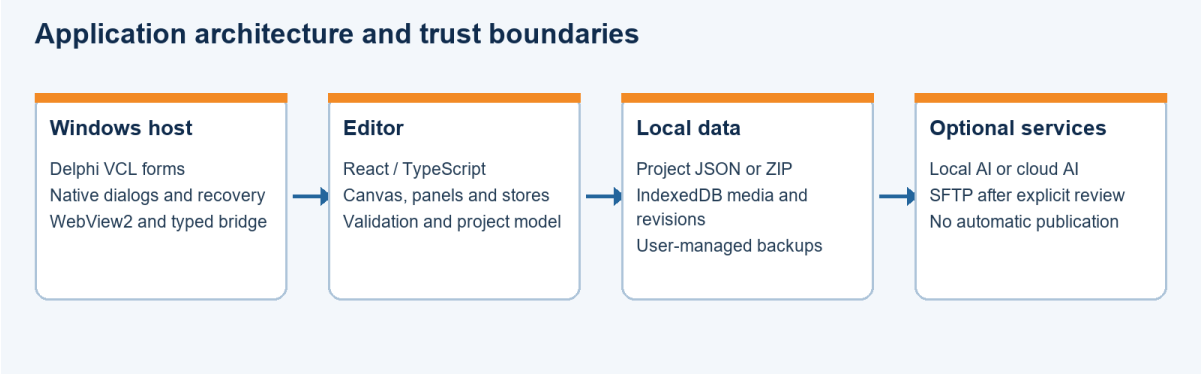
Delphi Web Studio Runtime Flow

A source based walkthrough from launch to application close

Last changed: November 1, 2026

Source based illustrated edition



A guide to the current Delphi Web Studio application.

Contents

Scope and reading this flow 3
Lifecycle at a glance 3
1 Launch and main form 3
2 Browser boundary and React startup 4
3 Dashboard and project entry 4
4 Desktop bridge authorization 5
5 Editing and media 6
6 Save, revisions, backup and recovery 7
7 Website output 7
8 Optional AI and SFTP 8
9 Close and result interpretation 9

Scope and reading this flow

This document traces the implemented Delphi Web Studio runtime from the executable entry point through the editor, project operations, and shutdown. The sequence is based on the current local source. It describes conditional paths; it does not claim that every host, clean installation, or interruption has been tested.

Each Source line identifies the code responsible for the preceding description. The Engineering Guide explains module ownership and data structures. The User Guide describes controls.

Lifecycle at a glance

Launch: VCL creates the main form and checks for its adjacent editor files.

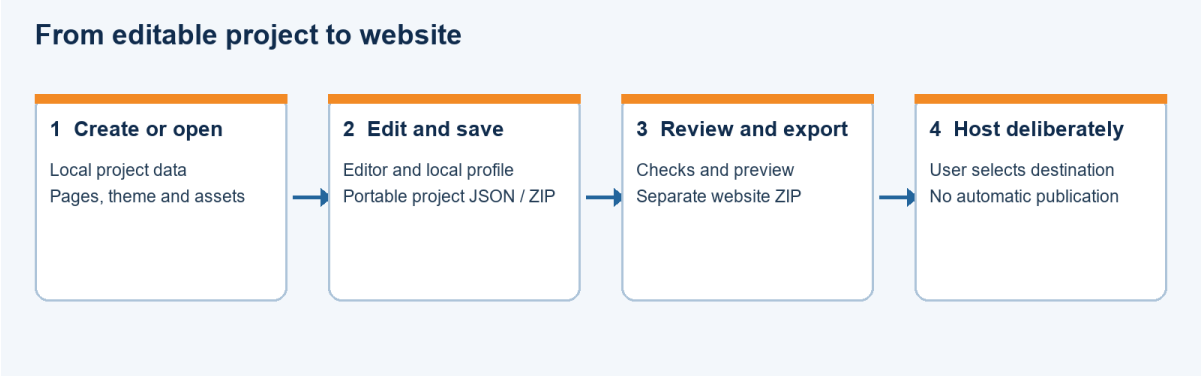


Figure 1. Typical lifecycle and separate publication decision.

Browser initialization: WebView2 maps the editor folder to the local virtual host, applies restrictions, and loads the React entry point.

Ready: React chooses the desktop router, initializes stores, and displays the dashboard.

Project branch: The user creates, opens, imports, or selects a local project. The editor renders the active page from configuration.

Persistence branch: Draft state, portable project files, media, and local revisions follow different storage paths.

Output branch: Review and export produce local website files. Optional AI or SFTP work follows separate explicitly chosen paths.

Close: Native form guards active operations and revokes its bridge session on destruction.

1 Launch and main form

1.1 Executable entry

Windows starts DelphiWebStudio.exe. The DelphiWebStudio.dpr program calls Application.Initialize, sets MainFormOnTaskbar, creates TMainForm, and enters Application.Run. The VCL message loop then dispatches the main form's native events. The form is configured maximized in its DFM, not by an assumed browser setting.

Source: desktop/DelphiWebStudio/DelphiWebStudio.dpr; desktop/DelphiWebStudio/WebStudio.Main.dfm.

1.2 Local editor check

TMainForm.FormCreate constructs the path editor\\index.html relative to ParamStr(0). If it is absent, it puts a missing-local-editor message in StatusLabel and returns without creating the WebView. If present, it calls EditorBrowser.CreateWebView. An exception changes the status rather than implying that the editor loaded.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, FormCreate.

1.3 WebView completion

EditorBrowserCreateWebViewCompleted checks the result and requires ICoreWebView2_3. It obtains WebView settings, enables web messages, maps the adjacent editor folder to webstudio.local with the configured access restriction, then navigates to https://webstudio.local/index.html. Failure in creation, settings, or mapping updates the status and exits before successful navigation.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, EditorBrowserCreateWebViewCompleted.

2 Browser boundary and React startup

2.1 Navigation and permissions

EditorBrowserNavigationStarting revokes and frees any existing desktop bridge session before it checks the destination. The host allows its local origin and cancels other top-level navigation. Frame navigation is cancelled and new-window requests are handled by the host. Permission handling permits a narrow local automatic-download case when the required WebView2 interface is available and denies other requests.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, EditorBrowserNavigationStarting, EditorBrowserFrameNavigationStarting, EditorBrowserNewWindowRequested and EditorBrowserPermissionRequested.

2.2 Script entry and routing

src/main.tsx adds a local-origin content security policy, obtains the root element, and renders App inside React StrictMode. App chooses HashRouter for the local desktop origin and BrowserRouter for other origins. AppRoutes wraps dashboard, editor, components, export, and settings routes in AppLayout and provides a not-found route. The root route renders Dashboard.

Source: src/main.tsx; src/App.tsx.

2.3 Ready state and local drafts

The project and configuration Zustand stores read their persisted browser state. Their guarded storage adapters preserve unreadable or incompatible values in the cases detected by the implementation. The editor store carries interactive state and session-only credential changes. IndexedDB-backed media and revisions are separate databases and are accessed by their corresponding modules.

Source: src/store/projectsStore.ts; src/store/configStore.ts; src/store/editorStore.ts; src/media/media-library.ts; src/project/project-revisions.ts.

3 Dashboard and project entry

3.1 New local project

Dashboard offers a blank project, a selected starter, or a description-based local starter. The create action produces project configuration and an identity in the local project store, selects it, and navigates to the editor. Choosing a local starter is not itself a cloud Gemini request. A local card represents a browser-profile draft, not proof of an independently saved project file.

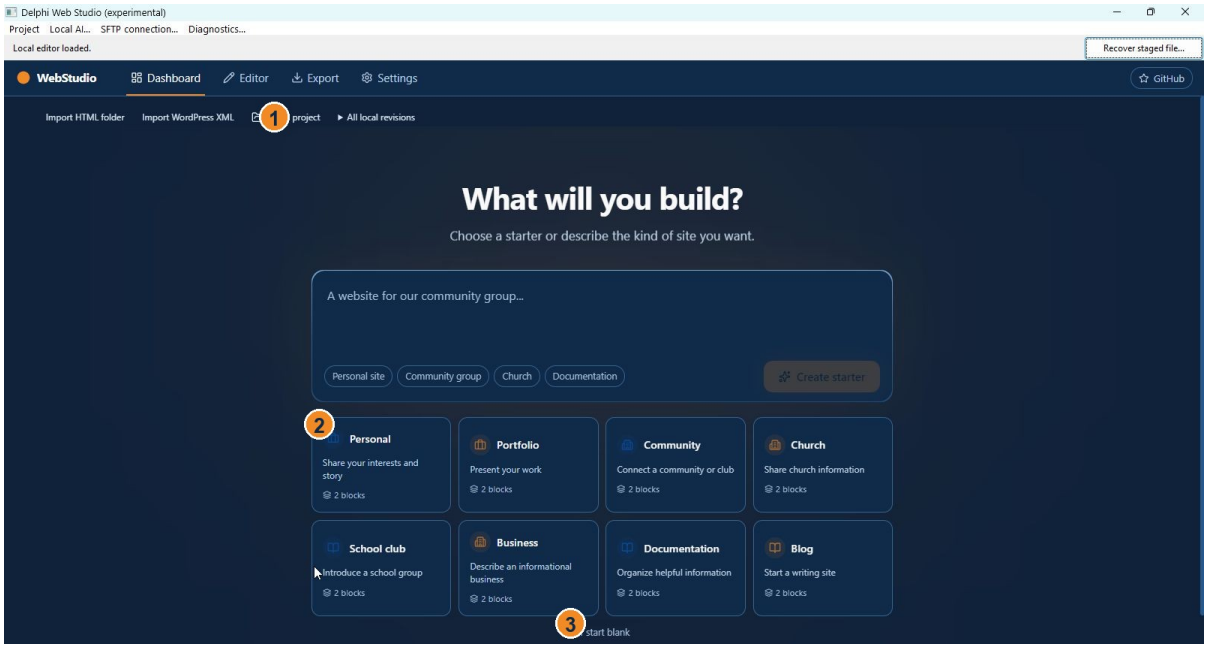


Figure 2. The dashboard appears after editor initialization.

Source: src/routes/Dashboard.tsx; src/lib/templates.ts; src/store/projectsStore.ts.

3.2 Opening a portable project

ProjectFileCommands receives a selected file and enters its open path. It requests the desktop project.open capability, reads and decodes the supported input, parses project data, and checks associated media. The feature and schema validation in project-file.ts decide whether the data can be used or migrated. An accepted file is opened as a separate draft rather than silently modifying the chosen source.

The operation can fail on invalid data, unsupported format, missing or damaged media, a denied bridge request, or changed active project state. The error is reported; the original selected file should be retained. Unsupported block data can be preserved but does not thereby become exportable.

Source: src/project/ProjectFileCommands.tsx, openProjectFile; src/project/project-file.ts; src/desktop/native-bridge.ts.

3.3 Import branches

Static HTML import first validates selected-file limits, asks for project.open authorization, reads supported text and image candidates, and builds a preview. A later confirmation stages media and adds a separate project. The handler checks whether the original active project changed between preview and confirmation; failure can trigger staged-media cleanup.

WordPress import reads a supported WXR XML export into a preview and then creates a separate converted project. The converter limits input, rejects prohibited XML constructs, and retains source/report data as designed. Neither branch reconstructs arbitrary scripts, themes, plugins, or original server behavior.

Source: src/project/ProjectFileCommands.tsx, previewStaticHtml, createStaticProject and previewWordPress; src/project/static-html-import.ts; src/project/wordpress-import.ts.

4 Desktop bridge authorization

4.1 Browser request

For a local desktop action, requestDesktopAction checks the local host and validates the project identifier. It opens a versioned session for one capability, then sends a second request with the session ID. Each exchange listens for a

matching request ID and has a timeout. The authorization envelope does not carry the actual project file or a native filesystem path.

Source: src/desktop/native-bridge.ts; src/desktop/bridge-protocol.ts.

4.2 Native validation

EditorBrowserWebMessageReceived obtains the sender and message from WebView2. It rejects nonlocal sources, oversized payloads, malformed JSON and invalid fields. A session.open message maps the named capability to project.open, project.save, media.import, or site.export. TDesktopSession generates and holds the session grant. Subsequent requests pass through ValidateAndClaim before an acknowledgement is sent.

The session enforces project and capability binding and replay protection. Navigating the WebView or destroying the form revokes the session. A bridge acknowledgement authorizes the existing local operation; it is not evidence that a later save, import, or export has succeeded.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, EditorBrowserWebMessageReceived; desktop/bridge/WebStudio.Bridge.pas.

5 Editing and media

5.1 Configuration to canvas

The editor route combines the layout, selected page, component registry, canvas, and panels. The configuration store holds site and page data and undo/redo state; the editor store holds selection, view mode and interactive session state. Adding, removing or rearranging supported sections updates configuration. PropertiesPanel applies edits to the selected block. Preview and static export use separate rendering paths, so parity is not assumed.

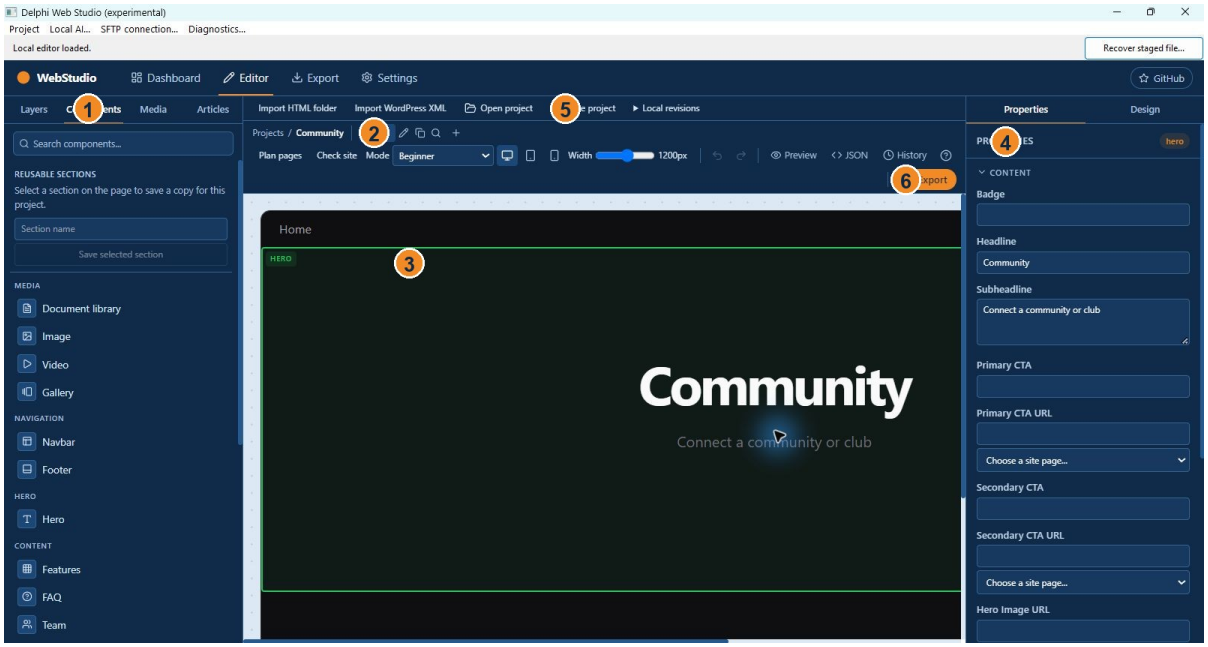


Figure 3. The editor renders an active project and page.

Source: src/routes/Editor.tsx; src/editor/EditorLayout.tsx; src/editor/Canvas.tsx; src/editor/PropertiesPanel.tsx; src/store/configStore.ts; src/blocks/registry.tsx; src/lib/export-html.ts.

5.2 Media staging

MediaPanel collects input files and user duplicate policy. ProjectMediaLibrary validates supported inputs, maintains project-scoped asset metadata and content in the webstudio-media IndexedDB database, and calculates content hashes for relevant operations. An import may be cancelled; a pending list can be resumed while the application remains open. Failure or cancellation is not a promise that every staged item was imported.

Source: src/media/MediaPanel.tsx; src/media/media-library.ts.

6 Save, revisions, backup and recovery

6.1 Snapshot and packaging

saveProject captures project state, requests project.save authorization for the active project, validates the project and required media, and serializes the manifest. When media is present the operation creates a ZIP with the manifest and assets; a media-free save can use JSON. The resulting Blob is sent through a browser download. If state changes while a snapshot is prepared, the saved snapshot can represent an earlier state.

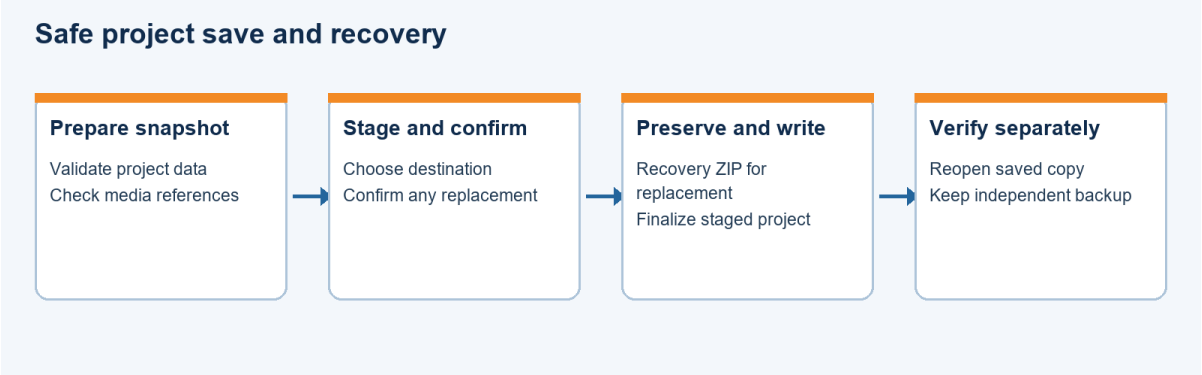


Figure 4. The project save and recovery branches.

Source: src/project/ProjectFileCommands.tsx, saveProject; src/project/project-file.ts; src/media/media-library.ts.

6.2 Native download and replacement

EditorBrowserDownloadStarting handles recognized local downloads and uses native destination dialogs and staging. The project-save implementation validates the staged file and guards the replacement path. Where an existing project is replaced, the user is asked to confirm and a recovery ZIP is created and checked before replacement. A failed or interrupted operation can leave recovery material or a staged file; a status message identifies what the user can inspect.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, EditorBrowserDownloadStarting; desktop/project/WebStudio.ProjectSave.pas.

6.3 Other recovery branches

ProjectRevisions saves and lists local snapshots in its own IndexedDB store; a revision is not automatically a portable file. BackupProjectItemClick selects a project file and creates a checked recovery ZIP. RestoreBackupItemClick validates the archive and writes a selected restored copy. RecoverStageButtonClick performs preliminary recognition of a retained staged download and writes a new name in the same folder; recognition does not prove the recovered project is complete.

Source: src/project/project-revisions.ts; desktop/DelphiWebStudio/WebStudio.Main.pas, BackupProjectItemClick, RestoreBackupItemClick and RecoverStageButtonClick.

7 Website output

7.1 Review and generate

The export route obtains the active configuration and project settings. Its local review path prepares website output, a list of files, and optional manifest and comparison material. If project data changes after review, the UI requires the user to review the current version again. The static exporter renders supported page blocks, paths, metadata and media into HTML or a site ZIP.

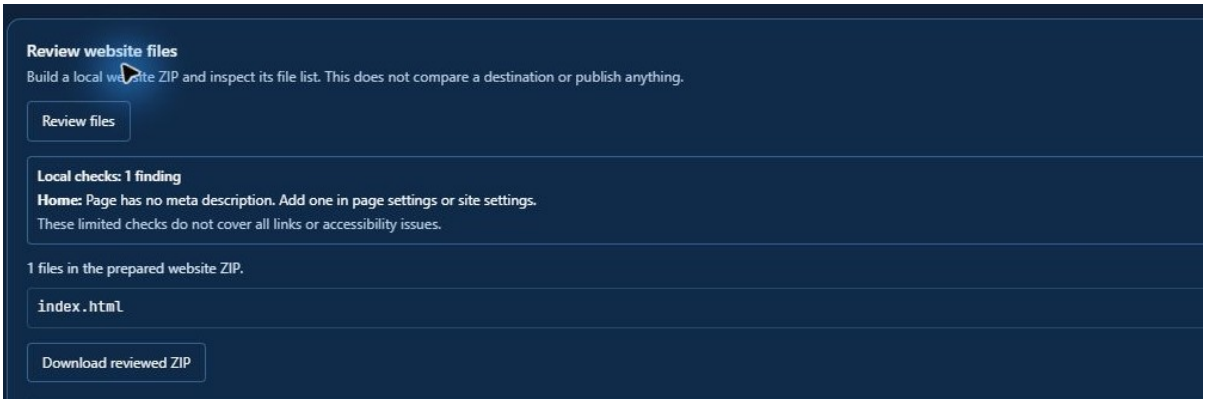


Figure 5. Local output review precedes any publication.

The project snapshot, local review, and generated website files are distinct artifacts. Unsupported blocks or missing media can stop export rather than being silently treated as valid. The generated site still requires testing for links, assets, host paths, forms, browser behavior and content.

Source: src/routes/Deploy.tsx, reviewWebsiteFiles and handleExport; src/lib/export-html.ts; src/lib/website-manifest.ts.

7.2 No implicit publication

Producing or downloading a site does not create a GitHub repository, push files, or turn on GitHub Pages. SFTP review and transfer are separate native paths. A real publication is outside this document's normal local flow and requires the owner's explicit instruction.

Source: src/routes/Deploy.tsx; desktop/DelphiWebStudio/WebStudio.SftpForm.pas; desktop/project/WebStudio.DeploymentExecutor.pas.

8 Optional AI and SFTP

8.1 Local AI worker

LocalAIItemClick first checks that no local AI task is active, that the WebView exists, and that an editor project is open. It reads the native dialog's selected runtime archive, model, brief and proposal type. It allocates a request ID, signals planning start, sets FLocalAIWorking, disables the menu, and starts an anonymous background thread.

The worker calls RunLocalSitePlan and queues a completion or failure callback to the UI thread. The callback clears the working flag, reenables the menu, and dispatches the corresponding local-plan event. A completed proposal is not an accepted project change: the editor must validate and present it for review.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, LocalAIItemClick; desktop/ai/WebStudio.LocalAIHost.pas; src/ai/LocalPlanReview.tsx.

8.2 Cloud AI and keys

Cloud generation is a separate optional network branch. Session state determines whether a newly entered Gemini key and cloud consent are available. A legacy key may be read from local storage if present. Key test and generation make network requests when explicitly invoked; a successful key test does not establish every model, billing or quota condition.

Source: src/routes/Settings.tsx; src/lib/generate-site.ts; src/store/editorStore.ts.

8.3 SFTP review and execution

The native SFTP dialog collects connection information. Connection test, inventory, destination backup, deployment review, preflight, execution and rollback are distinct operations implemented by desktop/project

modules. The executor requires a confirmed reviewed deployment. A failed transfer can leave a partially changed destination, so receipts, recovery material and explicit review matter.

Source: desktop/DelphiWebStudio/WebStudio.SftpForm.pas; desktop/project/WebStudio.SftpConnection.pas; WebStudio.DeploymentReview.pas; WebStudio.DeploymentPreflight.pas; WebStudio.DeploymentExecutor.pas; WebStudio.RollbackExecutor.pas.

9 Close and result interpretation

9.1 Closing guards

FormCloseQuery prevents close while FLocalAIWorking is true. For an active save or download it initially warns and blocks close; a subsequent close can interrupt the operation. FormDestroy revokes and frees the bridge session. A user should wait for the completion status and independently reopen important saved files.

Source: desktop/DelphiWebStudio/WebStudio.Main.pas, FormCloseQuery and FormDestroy.

9.2 What the sequence proves

This flow describes the implementation inspected on November 1, 2026. A compiled application, a completed local export, or a passing selected test is not proof that a clean Windows installation, all hosting environments, every recovery case, or real publication has been verified. The clean-Windows test and real publication remain deferred. No actual website was published to produce this document.